

# AULA 11 – COMPLEMENTO PRÁTICO

## PHP + MySQL (Sakila) e PHP + PostgreSQL

- ✓ Conteúdo **emerge da necessidade** de persistência
- ✓ Mantém o foco da SA original
- ✓ Prepara diretamente para o **CRUD da Aula 12**

### ◆ Contexto Profissional (continuidade da SA)

A empresa fictícia agora precisa **consultar dados reais armazenados no banco**, usando **PHP**, para gerar telas de listagem e relatórios simples.

## 1 Conexão PHP + MySQL (Banco Sakila)

### ✓ Por que Sakila?

O **Sakila** é um banco de testes padrão do MySQL, com:

- clientes
- filmes
- categorias
- locações

➡ Ideal para **consultas reais**, sem criar tabelas do zero.

### 📌 Exemplo 1 – Conectando PHP ao MySQL (PDO)

```
PHP
connect_mysql.php
<?php
$host = "localhost";
$dbname = "sakila";
$user = "root";
$pass = "";

try {
```

```

        $conexao = new PDO(
            "mysql:host=$host;dbname=$dbname;charset=utf8mb4",
            $user,
            $pass
        );

        echo "Conexão com MySQL realizada com sucesso!";
    } catch (PDOException $e) {
        echo "Erro na conexão: " . $e->getMessage();
    }
?>

```

## PHP + PostgreSQL (mesma lógica, outro banco)

### Conexão com PostgreSQL (PDO)

```

PHP
connect_postgress.php
<?php

$host = "localhost";
$dbname = "empresa";
$user = "postgres";
$pass = "123";

try {
    $conexao = new PDO(
        "pgsql:host=$host;dbname=$dbname",
        $user,
        $pass
    );
    echo "Conexão com PostgreSQL realizada!";
} catch (PDOException $e) {
    echo "Erro: " . $e->getMessage();
}
?>

```

### Explicação didática

- **PDO** → padrão profissional (serve para MySQL e PostgreSQL)
- **try/catch** → evita que o sistema quebre
- **charset=utf8** → evita problemas com acentuação

 **Relacionar com mercado:** praticamente todo sistema PHP profissional usa PDO.

---

## 2 SELECT com PHP + MySQL (Sakila)

 **Exemplo 2 – Listar clientes do Sakila**

```
PHP
select.php
<?php
require_once 'connect_mysql.php';

$sql = "SELECT customer_id, first_name, last_name, email
        FROM customer
        LIMIT 10";

//stmt = statement refere-se a um objeto PDOStatement no contexto do PDO
$stmt = $conexao->prepare($sql);
$stmt->execute();

$clientes = $stmt->fetchAll(PDO::FETCH_ASSOC);

foreach ($clientes as $cliente) {
    echo "ID: {$cliente['customer_id']}<br>";
    echo "Nome: {$cliente['first_name']} {$cliente['last_name']}<br>";
    echo "Email: {$cliente['email']}<hr>";
}
?>
```

 **O que o aluno precisa entender**

- `prepare()` → segurança (anti SQL Injection)
  - `execute()` → executa o SQL
  - `fetchAll()` → traz vários registros
  - `foreach` → integração com Aula 06 e 07
- 

## 3 SELECT com WHERE (Filtro)

 **Exemplo 3 – Buscar cliente por ID**

```

PHP
<?php
require_once "connect_mysql.php";

$id = 5;

$sql = "SELECT first_name, last_name
        FROM customer
        WHERE customer_id = :id";

$stmt = $conexao->prepare($sql);
$stmt->bindParam(":id", $id);
$stmt->execute();

$cliente = $stmt->fetch(PDO::FETCH_ASSOC);

echo "Cliente: {$cliente['first_name']} {$cliente['last_name']}";
?>

```

## 4 INSERT com PHP + MySQL (exemplo didático)

⚠ O Sakila não é ideal para INSERT em produção, mas o exemplo é **pedagógico**.

```

PHP
<?php
require_once "connect_mysql.php";

$sql = "INSERT INTO actor (first_name, last_name)
        VALUES (:nome, :sobrenome)";

$stmt = $conexao->prepare($sql);
$stmt->bindValue(":nome", "JOAO");
$stmt->bindValue(":sobrenome", "SILVA");

$stmt->execute();

echo "Ator inserido com sucesso!";
?>

```

## 5 UPDATE e DELETE (conceito aplicado)

PHP

```
<?php
```

```
require_once "connect_mysql.php";
```

```
$sql = "UPDATE actor  
      SET last_name = 'SOUZA'  
      WHERE actor_id = 201";
```

```
$conexao->exec($sql);
```

```
?>
```

```
<?php
```

```
require_once "connect_mysql.php";
```

```
$sql = "DELETE FROM actor WHERE actor_id = 201";  
$conexao->exec($sql);  
?>
```